

บทที่ 5

การเขียนดีไวซ์ไดรเวอร์

อุปกรณ์แต่ละชนิดย่อมมีลักษณะการทำงานที่แตกต่างกันออกไป ซึ่งโอเอส (OS) มีหน้าที่ควบคุมและติดต่อกับอุปกรณ์ต่าง ๆ ทุกประเภท ดังนั้นโอเอสจึงจำเป็นต้องรู้ลักษณะการทำงานของอุปกรณ์ทุกประเภท เพื่อจะได้สามารถควบคุมการทำงานของอุปกรณ์แต่ละประเภทได้ถูกต้อง แต่หากเรานำเอาโปรแกรมที่เป็นส่วนควบคุมอุปกรณ์ทั้งหมดมาใส่รวมไว้ในตัวโอเอส จะทำให้โอเอส มีขนาดใหญ่มาก ผู้ออกแบบโอเอสจึงมีความคิดที่จะแยกเอาส่วนที่ควบคุมอุปกรณ์ต่าง ๆ ออกจากตัวโอเอส ซึ่งโปรแกรมที่แยกออกมานี้ เรียกว่า ดีไวซ์ไดรเวอร์ (Device Driver) เมื่อโอเอสต้องการที่จะติดต่อกับอุปกรณ์ใด โอเอสจะทำการติดต่อผ่านทางดีไวซ์ไดรเวอร์ชนิดนั้นแล้วดีไวซ์ไดรเวอร์จะทำการติดต่อกับอุปกรณ์จริงเอง

5.1 ความหมายของดีไวซ์ไดรเวอร์

ดีไวซ์ไดรเวอร์เป็นโปรแกรมพิเศษที่ถูกโหลดเมื่อระบบทำงาน มีหน้าที่ในการเชื่อมต่อการทำงานระหว่างแอปพลิเคชัน (Application) ของผู้ใช้งานกับอุปกรณ์ฮาร์ดแวร์ (Hardware) ซึ่งควรออกแบบให้มีความยืดหยุ่นในการใช้งาน ไม่จำกัดการใช้งานฮาร์ดแวร์ โดยให้ตัวแอปพลิเคชันเป็นตัวเลือกการทำงานเอง ส่วนดีไวซ์ไดรเวอร์ทำหน้าที่เพียงจัดการทรัพยากร (Resource) ภายในของคอมพิวเตอร์ให้เหมาะสมแก่การใช้งาน

5.2 การสร้างไฟล์ของอุปกรณ์

ลินุกซ์เคอร์เนล (Linux Kernel) จะทำการเรียกตัวดีไวซ์ไดรเวอร์ด้วยเลขเมเจอร์ (Major Number) และ เลขไมเนอร์ (Minor Number) โดยอุปกรณ์ที่ถูกควบคุมด้วยดีไวซ์ไดรเวอร์ตัวเดียวกัน อุปกรณ์เหล่านั้นก็จะมีเลขเมเจอร์เท่ากัน และเลขไมเนอร์จะใช้ในการจำแนกอุปกรณ์ภายในเลขเมเจอรื้นั้น ๆ เช่น มีฟลอปปีดิสก์ (Floppy disk) 2 ตัวในเครื่อง ก็จะมีเลขเมเจอร์เป็น 2 โดยที่ฟลอปปีดิสก์ตัวแรกจะมีเลขไมเนอร์เป็น 0 และ อีกตัวหนึ่งเป็น 1 ซึ่งเคอร์เนลจะต้องทราบก่อนว่า มีอุปกรณ์นั้นมียู่จริงก่อนที่จะนำไดรเวอร์ของตัวนั้นมาใช้งาน โดยเมื่อทำการบูตระบบขึ้นมาแล้ว เคอร์เนลจะทำการสร้างไฟล์พิเศษขึ้นมาสำหรับแต่ละอุปกรณ์ในระบบโดยเก็บไว้ที่ไดเรกทอรี (Directory) /dev ซึ่งไฟล์พิเศษนี้ จะถูกสร้างขึ้นด้วยคำสั่ง `mknod`

`mknod /dev/device_name device_type major_number minor_number`

- `device_name` คือ ชื่อของไฟล์
- `device_type` คือ ชนิดของอุปกรณ์ ว่าเป็น คาแร็กเตอร์ดีไวซ์ (c) หรือ เป็นบล็อกดีไวซ์ (b)
- เลขเมเจอร์ซึ่งสามารถดูได้จากไฟล์ชื่อ `Documentation/devices.txt` ในซอสโค้ดของเคอร์เนลที่ใช้อยู่

5.3 ชนิดของอุปกรณ์

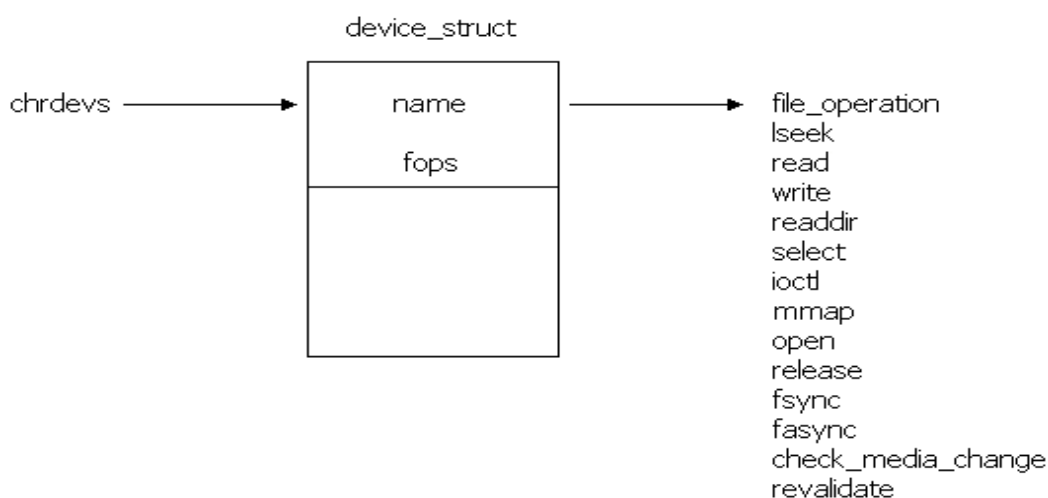
5.3.1 คาแร็กเตอร์ดีไวซ์ (Character Device)

อุปกรณ์ที่กระทำกับข้อมูลอินพุตและเอาต์พุต โดยไม่ใช้บัฟเฟอร์ (Buffer) ซึ่งอีกนัยหนึ่งคือทำการแบบสตีมมิ่ง ซึ่งรายชื่อของอุปกรณ์เหล่านี้ จะอยู่ในโครงสร้างข้อมูล `device_struct` ซึ่งถูกชี้ด้วยเวกเตอร์ (Vector) ที่ชื่อว่า `chrdev` ที่มีดัชนี (Index) เป็นเลขแมเจอร์ และอีกฟิลด์ (Field) หนึ่งของโครงสร้างข้อมูลนี้ คือ พอยน์เตอร์ (Pointer) ที่ชี้ไปยังฟังก์ชันการทำงานต่าง ๆ กับไฟล์ ซึ่งอยู่ในโครงสร้างข้อมูล `file_operations` โดยที่ใช้ฟังก์ชันพื้นฐานทำงานกับอุปกรณ์ได้เหมือนทำงานกับไฟล์ทั่วไป (มองอุปกรณ์เป็นไฟล์) ฟังก์ชันพื้นฐาน ได้แก่ `open()`, `read()`, `write()` และ `close()`

ซึ่งรูทีน (Routine) การทำงานของอุปกรณ์ที่ระบุในโครงสร้างข้อมูล `file_operations` เป็นดังนี้

```
struct file_operations {
    int (*lseek)    (struct inode *, struct file *, off_t, int);
    int (*read)     (struct inode *, struct file *, char *, int);
    int (*write)    (struct inode *, struct file *, const char *, int);
    int (*readdir)  (struct inode *, struct file *, struct dirent *, filldir_t);
    int (*select)   (struct inode *, struct file *, int, select_table *);
    int (*ioctl)    (struct inode *, struct file *, unsigned int, unsigned
long);
    int (*mmap)     (struct inode *, struct file *, struct vm_area_struct);
    int (*open)     (struct inode *, struct file *);
    int (*release)  (struct inode *, struct file *);
};
```

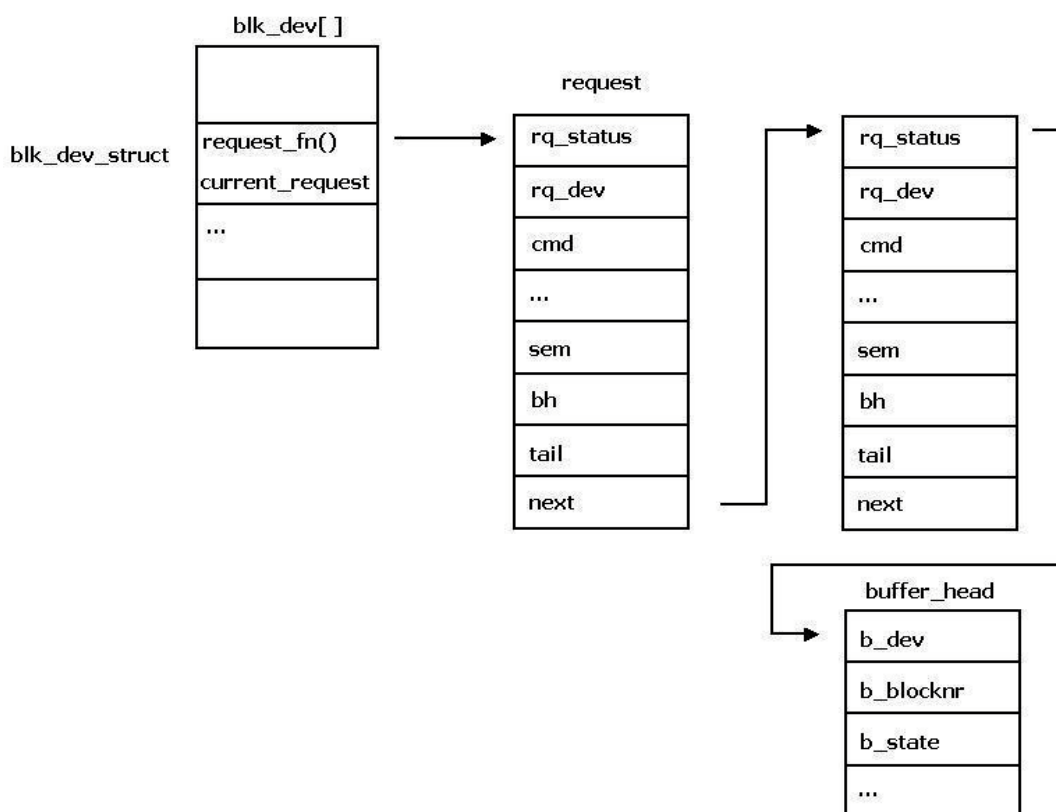
เมื่อแอปพลิเคชันต้องการใช้อุปกรณ์ จะต้องเปิดไฟล์ของอุปกรณ์โดยกระทำ ดังนี้



รูปที่ 5-1: โครงสร้างข้อมูลของ Character Device

5.3.2 บล็อกดีไวซ์ (Block Device)

อุปกรณ์ที่กระทำกับข้อมูลอินพุต (Input) และเอาต์พุต (Output) เป็นบล็อก โดยกระทำผ่านบัฟเฟอร์ เพื่อให้สามารถเข้าถึงข้อมูลได้เร็วขึ้น ซึ่งส่วนใหญ่จะใช้เป็นสื่อในการส่งข้อมูล



รูปที่ 5-2: โครงสร้างข้อมูลของ Block Device

5.3.3 เน็ตเวิร์กดีไวซ์ (Network Device)

อุปกรณ์ที่ทำหน้าที่รับหรือส่งแพ็กเก็ตข้อมูล ซึ่งทำงานร่วมกับบีเอสดีซ็อกเก็ต (BSD Socket) ส่งผ่านข้อมูลโดยใช้ซ็อกเก็ตบัฟเฟอร์ (Socket Buffer) เพื่อประมวลผลตามมาตรฐานของโปรโตคอล (Protocol) ชั้นต่าง ๆ จนถึงการติดต่อกับอุปกรณ์ฮาร์ดแวร์จริง

5.4 การเขียนโปรแกรมดีไวซ์ไดรเวอร์บนระบบปฏิบัติการลินุกซ์

การเขียนดีไวซ์ไดรเวอร์ คือ การเขียนโปรแกรมเพื่อรันในส่วนของลินุกซ์เคอร์เนล ซึ่งไดรเวอร์จะรันบนเคอร์เนลโหมด (Kernel mode) โดยไดรเวอร์บนลินุกซ์จะถูกแบ่งออกเป็น 2 แบบคือแบบบิลท์อิน (Built-in) และแบบโมดูล (Module) ซึ่งจะต่างกันที่ช่วงเวลาที่ใช้เรียกการทำงาน

ลินุกซ์โมดูลเป็นเซตของฟังก์ชันและชนิดของข้อมูล ซึ่งสามารถจะคอมไพล์ (Compile) แยกจากตัวเคอร์เนลได้ และจะถูกเรียกใช้งานหลังจากที่บูตเคอร์เนลแล้ว โมดูลจะใช้ในการอิมพลีเมนต์ (Implement) ดีไวซ์ไดรเวอร์ ซึ่งจะใช้ติดต่อกับอุปกรณ์เฉพาะของไดรเวอร์นั้นเท่านั้น

5.4.1 ขั้นตอนการเขียนโปรแกรมดีไวซ์ไดรเวอร์

1. เรียนรู้การทำงานภายในของโอเอสที่ใช้งาน เช่น วินโดวส์ หรือ ลินุกซ์ เป็นต้น
2. เรียนรู้ว่าจะเขียนดีไวซ์ไดรเวอร์อย่างไรบนโอเอสที่ใช้งานอยู่ เช่น DDK, DDI หรือ DKI เป็นต้น
3. หา Tool มาใช้สำหรับการพัฒนาและการดีบั๊กในเคอร์เนลโหมด
4. ทำการเขียนดีไวซ์ไดรเวอร์บนเคอร์เนลโหมด ซึ่งเป็นไดรเวอร์พื้นฐานของฮาร์ดแวร์ที่เป็นส่วนติดต่อกับฮาร์ดแวร์และเอาต์พุต
5. ทำการเขียนแอปพลิเคชันในยูสเซอร์โหมด (User mode) ซึ่งสามารถใช้ฮาร์ดแวร์ผ่านทางดีไวซ์ไดรเวอร์ที่เขียนอยู่ในเคอร์เนลโหมดได้
6. ทำขั้นตอนที่ 1-4 ใหม่ สำหรับเมื่อมีการเปลี่ยนเป็นโอเอสตัวใหม่ ซึ่งควรที่จะสามารถรันโค้ดในข้อ 5 ได้

5.4.2 โปรแกรมโมดูล

แบ่งเป็น 2 ส่วนหลัก ๆ คือ

1. `int init_modules()` : เป็นฟังก์ชันที่ถูกเรียกใช้งานเมื่อมีการติดตั้งโมดูลเข้าไปในเคอร์เนล ใช้ในการติดตั้งค่าหรือเตรียมพร้อมการทำงานของโมดูลนั้น
 - การเรียกใช้งาน : `insmod modules.o`
2. `void cleanup_modules()` : เป็นฟังก์ชันที่ถูกเรียกใช้งานเมื่อต้องการกำจัดโมดูลออกจากเคอร์เนลของระบบ ใช้ในการจัดการให้ระบบมีสภาพเหมือนเดิม
 - การเรียกใช้งาน : `rmmod modules`

โครงสร้างของซอสโค้ดของโมดูลที่สามารถเรียกใช้งานได้จะต้องประกอบด้วย 2 ฟังก์ชันดังนี้

```
#include <linux/modules.h>
...
int init_modules() {
    /* Code to init the module */
}
...
void cleanup_modules() {
    /* Code to close the module */
}
```

5.4.3 การคอมไพล์ (Compliment)

gcc -c modules.c

ที่ต้องเป็น -c นั้น เนื่องจากต้องการทำให้เป็นออบเจกต์ไฟล์ (Object file) เพื่อที่จะสามารถเพิ่มหรือลบโมดูลใดเวอร์ก็ได้ เพราะเคอร์เนลไม่ได้รันเป็นโปรแกรมอิสระ และไม่ต้องทำการลิงก์ฟังก์ชันภายนอก

5.5 คาเร็กเตอร์ดีไวซ์ไครเวอร์

การเขียนโปรแกรมไครเวอร์ของคาเร็กเตอร์ดีไวซ์ เป็นแบบที่ง่ายที่สุด เนื่องจากไม่มีการเกี่ยวข้องกับบัฟเฟอร์ โดยไครเวอร์จะทำงานในลักษณะของโมดูล คือ สามารถโหลดได้เมื่อเคอร์เนลต้องการใช้งาน ซึ่งทำงานเหมือนเป็นอุปกรณ์จริง

เคอร์เนลจะใช้เลขเมเจอร์เพื่อกำหนดไครเวอร์ที่จะใช้กับอุปกรณ์ และดีไวซ์ไครเวอร์จะใช้เลขไมเนอร์ในการแยกแยะอุปกรณ์แต่ละชิ้น ซึ่งการจะเข้าถึงเลขเมเจอร์และเลขไมเนอร์ได้นั้น ต้องใช้คำสั่ง inode -> i_rdev เพราะเลขทั้ง 2 ตัวนั้น เก็บไว้ในฟิลด์ i_rdev ที่อยู่ในโครงสร้าง inode โดยไครเวอร์จะมีอาร์กิวเมนต์ (Argument) รับพอยน์เตอร์ที่ชี้ไปยังโครงสร้างข้อมูล inode

5.5.1 การเพิ่มและลบโมดูลไครเวอร์ออกจากระบบ

การเพิ่มโมดูลไครเวอร์ทำได้โดยใช้ฟังก์ชัน ซึ่งกำหนดไว้ในไฟล์ < linux/fs.h > ฟังก์ชันคือ

int register_chrdevs (unsigned int major, const char *name, struct file_operations *fops);

โดย name คือ ชื่อของอุปกรณ์

fops คือ พอยน์เตอร์ชี้ไปยังโครงสร้างข้อมูล file_operations

ซึ่งฟังก์ชันจะส่งค่ากลับเป็นจำนวนเต็มลบ ถ้ามี ERROR เกิดขึ้น เป็นศูนย์หรือจำนวนเต็มบวก ถ้าทำงานสำเร็จ

การลบโมดูลไครเวอร์ออกจากระบบ จะเป็นการยกเลิกเลขเมเจอร์ที่ใช้งาน เพื่อให้ไครเวอร์อื่นสามารถนำเลขเมเจอร์ไปใช้งานได้ อีก ฟังก์ชัน คือ

int unregister_chrdev (unsigned int major, const char *name);

เคอร์เนลจะทำการเปรียบเทียบเลขเมเจอร์กับชื่อของอุปกรณ์ ในกรณีที่ตรงกันหรือเลขเมเจอร์มีค่ามากกว่าค่าที่กำหนดได้ จะส่งค่ากลับเป็น -EINVAL ในกรณีที่ไม่สามารถจะ unregister โมดูลได้ ตัวโปรเซส (Process) /proc/devices จะแสดงข้อผิดพลาดเมื่อทำการอ่านครั้งต่อไป เรียกว่า “Oops” ซึ่งเป็นข้อความที่แสดงเมื่อเคอร์เนลพยายามจะแอ็กเซส (Access) แอดเดรส (Address) ที่ไม่ถูกต้อง โดยต้องไม่สับสนกับโหนด (node) ที่เป็นชื่อของอุปกรณ์ออกจากระบบด้วย

5.5.2 ฟังก์ชันที่ใช้ทำงานกับอุปกรณ์ (ไฟล์)

เคอร์เนลจะทำงานกับอุปกรณ์ผ่านรูทีนของไดรเวอร์ที่เก็บไว้ในโครงสร้างข้อมูล `file_operations` โดยในกรณีที่ฟังก์ชันใดไม่ได้ทำงานกับอุปกรณ์ จะถูกเซตค่าเป็น `NULL` และจะส่งค่ากลับเป็น 0 เมื่อทำงานสำเร็จ หรือจำนวนเต็มลบเมื่อเกิด `ERROR` ขึ้น

สรุปการทำงานของฟังก์ชัน ดังต่อไปนี้

int (*lseek) (struct inode *, struct file *, off_t, int);

ใช้เปลี่ยนตำแหน่งในการอ่านหรือเขียนข้อมูลลงไฟล์ โดยค่าที่ส่งกลับจะเป็นจำนวนเต็มบวก หรือจุดออฟเซต (Offset) จากจุดเริ่มต้นไฟล์ ถ้าไม่มีการระบุฟังก์ชันนี้ จะทำให้ไดรเวอร์ไม่สามารถกำหนดตำแหน่งบนข้อมูลของอุปกรณ์ได้

int (*read) (struct inode *, struct file *, char *, int);

ใช้เอ็กเซตข้อมูลจากอุปกรณ์ ถ้ามีการกำหนดฟังก์ชันนี้เป็น `NULL` จะทำให้ฟังก์ชันของระบบ `read` ไม่สามารถทำงานได้ จะเกิด `ERROR -EINVAL` โดยค่าที่ส่งกลับจะเป็นจำนวนไบต์ที่อ่านได้ เมื่อทำงานสำเร็จ

int (*write) (struct inode *, struct file *, const char *, int);

ฟังก์ชันจะส่งข้อมูลไปยังอุปกรณ์ โดยค่าที่จะส่งกลับจะเป็นจำนวนไบต์ที่เขียนได้

int (*readdir) (struct inode *, struct file *, void *, filldir_t);

ฟังก์ชันนี้ ควรจะกำหนดให้เป็น `NULL` สำหรับไฟล์ `node` ของอุปกรณ์ เนื่องจากปกติจะใช้อ่านข้อมูลในไดเรกทอรี (Directory)

int (*select) (struct inode *, struct file *, int, select_table *);

ใช้โดยโปรแกรมเพื่อสอบถามว่า อุปกรณ์สามารถจะอ่านหรือเขียนได้หรือไม่ โดยถ้ามีการกำหนดฟังก์ชันนี้เป็น `NULL` จะหมายถึง สามารถทำได้ทั้ง 2 อย่าง ซึ่งขึ้นอยู่กับชนิดของอุปกรณ์ด้วย โดยค่าที่ส่งกลับจะเป็นการบอกว่า สามารถอ่านหรือเขียนได้ตามที่กำหนดหรือไม่

int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long);

ใช้สำหรับการออกคำสั่งเฉพาะของอุปกรณ์ เช่น การฟอร์แมต (Format) ดิสก์ โดยไม่มีการอ้างอิงไปยังตาราง `fops` โดยค่าที่ส่งกลับจะเป็นจำนวนเต็มบวก

int (*mmap) (struct inode *, struct file *, struct vm_area_struct *);

ใช้แมป (Map) พื้นที่หน่วยความจำของอุปกรณ์และหน่วยความจำของโปรเซส

int (*open) (struct inode *, struct file *);

ใช้เมื่อต้องการจะเริ่มการทำงานกับอุปกรณ์ โดยถ้ามีการกำหนดฟังก์ชันนี้เป็น `NULL` จะทำให้การเรียกใช้งานฟังก์ชันสำเร็จเสมอ แต่ไม่มีค่าส่งกลับ

int (*release) (struct inode *, struct file *);

ใช้งานเมื่อไฟล์ดีไวซ์ไดรเวอร์ (`/dev/...`) ของอุปกรณ์ถูกปิด

int (*fsync) (struct inode *, struct file *);

รีเซ็ต (Reset) อุปกรณ์ให้อยู่ในสถานะพร้อมทำงาน

```
int (*fasync) (struct inode *, struct file *, int);
```

ใช้แจ้งให้อุปกรณ์ทราบถึงสถานะแฟล็ก (Flag) FASYNC โดยถ้ามีการกำหนดฟังก์ชันนี้เป็น NULL หมายถึงอุปกรณ์ไม่สนับสนุนการทำงาน

```
int (*check_media_change) (kdev_t dev);
```

ใช้งานกับอุปกรณ์ที่เป็นบล็อกดีไวซ์ซึ่งฟังก์ชันนี้จะใช้งานโดยเคอร์เนลเพื่อตัดสินใจว่า มีการเปลี่ยนสื่อเก็บข้อมูลหลังจากการทำงานครั้งสุดท้ายหรือไม่

```
int (*revalidate) (kdev_t dev);
```

ใช้งานกับอุปกรณ์ที่เป็นบล็อกดีไวซ์ เพื่อจัดการกับหน่วยความจำแคช (Cache)

โค้ดด้านล่างเป็นการประกาศโครงสร้างข้อมูลระบุทีนสำหรับทำงานกับอุปกรณ์ schd (Special Character Device)

```
static struct file_operations schd_ops= {
    schd_lseek,
    schd_read,
    schd_write,
    NULL,           // schd_readdir
    NULL,           // schd_select
    schd_ioctl,
    NULL,           // schd_mmap
    schd_open,
    schd_release,   // Nothing more, fill with NULLS
};
```

5.5.3 การเปิดและปิดไฟล์ดีไวซ์ไดรเวอร์

5.5.3.1 ขั้นตอนการเปิดไฟล์ดีไวซ์ไดรเวอร์ เพื่อเริ่มต้นทำงาน

ฟังก์ชัน open จะทำการเซตอัพ (Setup) ทรัพยากรต่าง ๆ เพื่อเตรียมติดต่อกับอุปกรณ์ โดยจะใช้มาโคร (Macro) MOD_INC_USE_COUNT เป็นดัชนีระบุว่ายังมีการใช้งานเพื่อป้องกันการลบโครงสร้างข้อมูลก่อนปิดไฟล์ โดยฟังก์ชัน open จะทำงานดังนี้

- ตรวจสอบอุปกรณ์ว่าอยู่ในสภาพพร้อมที่จะทำงานหรือไม่
- เซตอัพอุปกรณ์ถ้าเพิ่งจะเริ่มเปิดเป็นครั้งแรก
- หาค่าของเลขเมเจอร์และอัปเดต (Update) โครงสร้างข้อมูล f_op
- กำหนดโครงสร้างข้อมูลที่เป็นขึ้นมา
- เพิ่มค่าตัวแปรเก็บจำนวนครั้งที่ใช้งาน

สิ่งแรกที่ไดรเวอร์ทำคือ การเอ็กรหัสเลขไมเนอร์ที่ฟิลด์ inode->i_rdev โดยอุปกรณ์แต่ละชนิดจะทำงานไม่เหมือนกันถึงแม้จะใช้ไดรเวอร์ตัวเดียวกัน ซึ่งที่จริงแล้วไดรเวอร์ไม่รู้จักอุปกรณ์ที่ต้องทำงานด้วย เพียงแต่ใช้เลขเมเจอร์และเลขไมเนอร์เป็นตัว

แบ่งอุปกรณ์ โดยผู้ใช้งานสามารถกำหนดการทำงานของแต่ละอุปกรณ์ให้ทำงานแตกต่างกันจากการกำหนดชื่อให้ต่างกัน

ตัวอย่างโค้ดของฟังก์ชัน open ต่อไปนี้ แสดงขั้นตอนในการเซตอัพข้อมูลของอุปกรณ์ที่จำเป็น โดยมีการใช้มาโคร TYPE และ NUM แอ็กเซสบิตข้อมูลจากเลขไมเนอร์

```
int schd_open (struct inode *inode, struct file *filp);
{
    int type = TYPE(inode -> i_rdev);
    int num = NUM(inode -> i_rdev);
    Schd_Dev *dev;          /* Device information */

    /* Manage peculiar types first */
    if (type) {
        if (type > SCHD_MAX_TYPE) return -ENODEV;
        filp -> f_op = schd_fop_array[type];

        /* Dispatch to specific open */
        return filp -> f_op -> open(inode, filp);
    }

    /* Type 0, Check the device number */
    if (num >= schd_nr_devs) return -ENODEV;
    dev = &schd_devices[num];

    /* Trim to 0 the length of the device if open was write-only */
    if ( (filp -> f_flags & O_ACCMODE) == O_WRONLY)
        schd_trim(dev);          /* Ignore errors */

    /* And use filp -> private_data to point to the device data */
    filp -> private_data = dev;

    MOD_INC_USE_COUNT;
    Return 0;                  /* Success */
}
```

ข้อมูลเกี่ยวกับอุปกรณ์จะเก็บไว้ในที่โครงสร้างข้อมูล Schd_Dev จำนวนของอุปกรณ์ทั้งหมดที่มีชนิดเดียวกัน คือ schd_nr_devs และอาร์เรย์ (Array) ของพอยน์เตอร์ที่ชี้ไปยังพื้นที่หน่วยความจำ Schd_Dev คือ schd_devices[]

จากโค้ดตัวอย่างข้างต้นนี้ ไม่มีส่วนสำหรับทำงานบนอุปกรณ์จริง สิ่งที่ได้ทำงานเป็นเพียงการรีเซตข้อมูลของอุปกรณ์ให้เหลือ 0 ไบต์ เมื่อเริ่มมีการใช้ฟังก์ชัน open เท่านั้น

5.5.3.2 ขั้นตอนการปิดไฟล์ดีไวซ์ไครเวอร์ เมื่อเลิกใช้งาน

- ลดจำนวนการใช้งานลง
- ยกเลิกการใช้งานโครงสร้างข้อมูลต่าง ๆ ที่กำหนดขึ้นมาเมื่อใช้ฟังก์ชัน open
- ปิดอุปกรณ์

ตัวอย่างโค้ดของการปิดการใช้งานอุปกรณ์

```
void schd_release (struct inode *inode, struct file *filp);
{
    MOD_DEC_USE_COUNT;
}
```

การลดจำนวนครั้งการใช้งานลงเป็นสิ่งสำคัญมาก เนื่องจากเคอร์เนลไม่สามารถจะลบหรือปิดการใช้งานไฟล์ดีไวซ์ไครเวอร์ออกไปจากพื้นที่ของเคอร์เนลได้ ถ้าจำนวนการใช้งานไม่เท่ากับ 0 โดยโครงสร้างข้อมูล file จะถูกสร้างขึ้นเมื่อมีการใช้ฟังก์ชัน open เท่านั้น ถ้าจำนวนการใช้งานเป็น 0 ซึ่งจะเกิดขึ้นเมื่อมีการทำลายโครงสร้างข้อมูล file จึงจะสามารถใช้ฟังก์ชัน release เพื่อปิดการใช้งานอุปกรณ์ได้ (ปิดไฟล์ไครเวอร์)

5.5.4 การอ่านและเขียนข้อมูลบนอุปกรณ์

การอ่านและเขียนข้อมูลบนไครเวอร์เป็นการส่งผ่านข้อมูลระหว่างพื้นที่บนหน่วยความจำ 2 ส่วน คือ

1. พื้นที่ของเคอร์เนล และ
2. พื้นที่ของแอปพลิเคชัน

ฟังก์ชัน read และ write ในการอ่านข้อมูลบนอุปกรณ์จะใช้ฟังก์ชันต่อไปนี้สำหรับการอ่านและเขียนข้อมูลไปยังพื้นที่การทำงานของแอปพลิเคชัน

- ฟังก์ชันเขียนข้อมูลไปยังอุปกรณ์

```
void memcpy_fromfs(void *to, const void *from, unsigned long count);
```

- ฟังก์ชันอ่านข้อมูลจากอุปกรณ์

```
void memcpy_tofs (void *to, const void *from, unsigned long count);
```

ฟังก์ชัน read และ write จะส่งค่ากลับเป็นจำนวนเต็มลบถ้ามี ERROR เกิดขึ้น หรือเป็นจำนวนไบต์ที่อ่านได้หรือเขียนได้สำเร็จ

ตัวอย่างโค้ดของฟังก์ชันอ่านข้อมูลของอุปกรณ์จากพื้นที่ของเคอร์เนล

ฟังก์ชัน read จะส่งค่ากลับดังนี้

- ถ้าส่งค่ากลับเท่ากับค่าที่ระบุ count แสดงว่าข้อมูลถูกอ่านหมดทั้งหมด
- ถ้าเป็นจำนวนเต็มบวกน้อยกว่าค่าที่ระบุ count แสดงว่าส่งผ่านข้อมูลได้เพียงบางส่วน ปกติแล้วแอปพลิเคชันจะพยายามอ่านให้ครบตามจำนวนที่กำหนด
- ถ้าค่าที่ส่งกลับเป็น 0 แสดงว่าถึงจุดสิ้นสุดไฟล์
- ถ้าส่งค่ากลับเป็นจำนวนเต็มลบ แสดงว่ามี ERROR เกิดขึ้น ชนิดของ ERROR จะระบุไว้ในไฟล์ <linux/errno.h>

```
read_write_t schd_read (struct inode *inode, struct file *filp, char *buf,
count_t count);
```

```
{
    Schd_Dev *dev = filp -> private_data; /* The first list item */

    int quantum = dev -> quantum;
    int qset = dev -> qset;
    int itemsize = quantum * qset; /* How many bytes in the list */

    unsigned long f_pos = (unsigned long) (filp -> f_pos);
    int item, s_pos, q_pos, rest;

    if (f_pos > dev -> size) return 0;
    if (f_pos + count > dev -> size) count = dev -> size - f_pos;

    /* Find list item, qset index, and offset in the quantum */
    item    = f_pos / itemsize;
    rest    = f_pos % itemsize;
    s_pos   = rest / quantum;
    q_pos   = rest % quantum

    /* Follow the list up to right position (defined elsewhere) */
    dev = schd_follow(dev, item);

    if (!dev -> data) return 0; /* Don't fill holes */
    if (!dev -> data[s_pos])    return 0;
    if (count > quantum - q_pos) count = quantum - q_pos;
    dev -> usage++; /* The following call may sleep */
    memcpy_tofs(buf, dev -> data[s_pos] + q_pos, count);
    dev -> usage--;

    filp -> f_pos += count;
    return count;
}
```

ตัวอย่างโค้ดของฟังก์ชันเขียนข้อมูลของอุปกรณ์ไปยังพื้นที่ของเคอร์เนล

ฟังก์ชัน write จะส่งค่ากลับดังนี้

- ถ้าส่งค่ากลับเท่ากับค่าที่กำหนด count แสดงว่าการเขียนข้อมูลทำได้สำเร็จ
- ถ้าเป็นจำนวนเต็มบวกน้อยกว่าค่าที่ระบุ count แสดงว่าเขียนข้อมูลได้เพียงบางส่วน
- ถ้าค่าที่ส่งกลับเป็น 0 แสดงว่าไม่มีการเขียนข้อมูล อาจจะเนื่องจากอุปกรณ์ถูกบล็อกไว้ (ไม่ถือว่าเป็น ERROR)
- ถ้าส่งค่ากลับเป็นจำนวนเต็มลบ แสดงว่ามี ERROR เกิดขึ้น เช่นเดียวกับฟังก์ชัน read

```
read_write_t schd_write(struct inode *inode, struct file *filp, const
char *buf, count_t count);
```

```
{
    Schd_Dev *dev = filp -> private_data;
    Schd_Dev *dptr;
    int quantum = dev -> quantum;
    int qset = dev -> qset;
    int itemsize = quantum * qset;
    unsigned long f_pos = (unsigned long) (filp -> f_pos);
    int item, s_pos, q_pos, rest;

    /* Find list item, qset index and offset in the quantum */
    item    = f_pos / itemsize;
    rest    = f_pos % itemsize;
    s_pos   = rest / quantum;
    q_pos   = rest % quantum;

    /* Follow the list up to the right position */
    dptr = schd_follow(dev, item);
    if (!dptr -> data) {
        dptr -> data = kmalloc(qset * sizeof(char *),
                                GFP_KERNEL);
        if (!dptr -> data) return -ENOMEM;
        memset(dptr -> data, 0, qset * sizeof(char *));
    }
    if (!dptr -> data[s_pos]) {
        dptr -> data[s_pos] = kmalloc(quantum,
                                        GFP_KERNEL);
        if (!dptr -> data[s_pos]) return -ENOMEM;
    }
}
```

```

/* Write to the end of quantum */
if (count > quantum - q_pos) count = quantum - q_pos;
dev -> usage++;      /* The following call may sleep */
memcpy_fromfs(dptr -> data[s_pos] + q_pos, buf, count);
dev -> usage--;

/* Update the size */
if (dev -> size < f_pos + count) (dev -> size = f_pos + count);
filp -> f_pos += count;
return count;
}

```

5.6 บล็อกดีไวซ์ไครเวอร์

5.6.1 การเพิ่มหรือลบไครเวอร์ออกจากเคอร์เนล

จะทำเช่นเดียวกับคาเร็กเตอร์ดีไวซ์ไครเวอร์ โดยเคอร์เนลจะใช้เลขเมเจอร์ในการจำแนกโมดูลไครเวอร์และใช้ฟังก์ชันต่อไปนี้ในการเพิ่มหรือลบไครเวอร์ออกจากเคอร์เนล

```
int register_blkdev(unsigned int major, const char *name, struct
file_operations *fops);
```

```
int unregister_blkdev(unsigned int major, const char *name);
```

5.6.2 ฟังก์ชันการอ่านและเขียนดีไวซ์ไครเวอร์

ฟังก์ชันที่ใช้สำหรับการอ่านและเขียนดีไวซ์ไครเวอร์ จะเป็นฟังก์ชันพื้นฐานที่ใช้งานโดยทั่วไปเรียกว่า “Strategic Routine” ซึ่งจะได้โครงสร้างข้อมูล file_operations ของอุปกรณ์ที่ชื่อว่า sbkd (Special block device) เป็นดังนี้

```

struct file_operations sbkd_fops = {
    NULL,          /* lseek: default */
    block_read,     /* Strategy routine */
    block_write,    /* Strategy routine */
    NULL,          /* sbkd_readdir */
    NULL,          /* sbkd_select */
    sbkd_ioctl,
    NULL,
    sbkd_open,
    sbkd_release,
    block_fsync,    /* Strategy routine */
    NULL,          /* sbkd_fasync */
    sbkd_check_change,
    sbkd_revalidate
};

```

และบล็อกดีไวซ์ยังมีโครงสร้างข้อมูลที่ทำหน้าที่เก็บลักษณะทางกายภาพไว้ด้วย คือ โครงสร้างข้อมูล blk_dev_struct ซึ่งเก็บไว้ในไฟล์ <linux/blkdev.h> โดยโครงสร้างเป็นดังนี้

```
struct blk_dev_struct {
    void (*request_fn) (void);
    struct request current_request;
    struct request plug;
    struct tq_struct plug_tq;
};
extern struct blk_dev_struct blk_dev[MAX_BLKDEV];
```

จากในโค้ดนั้น ยังมีอาร์เรย์ข้อมูลเก็บลักษณะของอุปกรณ์ นอกจาก blk_dev เก็บไว้ในไฟล์ drivers/block/ll_rw_block.c

ข้อแตกต่างที่สำคัญระหว่างคาแร็กเตอร์ดีไวซ์และบล็อกดีไวซ์ คือ คาแร็กเตอร์ดีไวซ์จะมีฟังก์ชันสำหรับอ่านหรือเขียนข้อมูลโดยตรงกับอุปกรณ์ ในขณะที่บล็อกดีไวซ์จะทำงานผ่านหน่วยความจำแคช (Cache) เมื่อโปรแกรมต่าง ๆ ต้องการข้อมูล ตัวโปรแกรมจะต้องทำการร้องขอไปยังหน่วยความจำแคช ถ้าไม่มีข้อมูลนั้นในแคช บล็อกดีไวซ์จะทำการเพิ่มโครงสร้างข้อมูล request ไปยังลิสต์ (List) ของโครงสร้างข้อมูล blk_dev_struct เพื่อทำการค้นหาข้อมูลที่โปรแกรมต้องการจากสื่อเก็บข้อมูล ดังนั้น ในโมดูล init จะต้องทำการกำหนดค่าของฟิลด์ request_fn ในโครงสร้างข้อมูล blk_dev_struct ด้วย ส่วนฟิลด์อื่น ๆ จะใช้งานภายในเคอร์เนล ซึ่งผู้เขียนไดรวเวอร์ไม่จำเป็นต้องใช้งาน

5.6.3 การยกเลิกการโหลดข้อมูลของไดรวเวอร์

การยกเลิกการโหลดข้อมูลจะทำเมื่อค่าของ counter เป็น 0 ซึ่งพื้นที่ของหน่วยความจำที่เก็บข้อมูลของอุปกรณ์ตัวนั้นจะถูกนำไปใช้ในงานอื่น โดยมีฟังก์ชันที่สนับสนุนการนำพื้นที่ไปใช้ คือ

1. ฟังก์ชัน check_media_change ทำการตรวจสอบว่า มีการเปลี่ยนอุปกรณ์หรือไม่ และ
2. ฟังก์ชัน revalidate ทำการอัปเดตแฟล็กสถานะหลังจากเปลี่ยนอุปกรณ์แล้ว

ตัวอย่างโค้ดต่อไปนี้ เป็นของฟังก์ชัน check_media_change โดยถ้ามีการยกเลิกการโหลดข้อมูลของดีไวซ์ไดรวเวอร์ จะได้ค่าที่ส่งกลับเป็น 1

```
int sbkd_check_change(kdev_t i_rdev)
{
    int minor = MINOR(i_rdev);
```

```

Sbkd_Dev *dev = sbkd_devices + minor;
if (minor >= sbkd_devs)      /* Paranoid */
    return 0;

PDEBUG("check_change for dev %I\n", minor);

if (dev -> data)
    return 0;                /* Still valid */
return 1;                    /* Expired */
}

```

และเมื่อมีการเปลี่ยนแปลงอุปกรณ์ ก็จำเป็นที่จะต้องอัปเดตแฟล็กสถานะของอุปกรณ์ ด้วย ซึ่งฟังก์ชันที่ใช้ในการอัปเดตคือ ฟังก์ชัน `revalidate` โดยฟังก์ชันนี้จะทำการจองพื้นที่หน่วยความจำเพื่อเก็บแฟล็กสถานะของอุปกรณ์ หากทำการจองพื้นที่ได้สำเร็จ จะได้ค่าที่ส่งกลับเป็น 0 ซึ่งตัวอย่างโค้ดต่อไปนี้ เป็นของฟังก์ชัน `revalidate`

```

int sbkd_revalidate(kdev_t i_rdev)
{
    Sbkd_Dev *dev = sbkd_devices + MINOR(i_rdev);

    PDEBUG("revalidate for dev &I\n", MINOR(i_rdev));

    if (dev -> data)
        return 0;
    dev -> data = vmalloc(dev ->size);
    if (!dev -> data)
        return -ENOMEM;
    return 0;
}

```

5.7 เน็ตเวิร์กไดรเวอร์

เน็ตเวิร์กไดรเวอร์มีการทำงานคล้ายกับบล็อกไดรเวอร์ คือ ทำการลงทะเบียนอุปกรณ์ไว้กับเคอร์เนลพร้อมทั้งกำหนดข้อมูลต่าง ๆ ให้แก่อุปกรณ์นั้น และจะทำงานเมื่อมีคำสั่งให้อ่าน / เขียนข้อมูลโดยทำผ่านฟังก์ชัน `request_fn` ซึ่งจะส่งผ่านข้อมูลระหว่างอุปกรณ์ส่งข้อมูลของเน็ตเวิร์กและซ็อกเก็ตบัฟเฟอร์เพื่อทำการประมวลผลมาตรฐานของโปรโตคอลชั้นต่าง ๆ อธิบายโดยให้ไดรเวอร์ใช้โปรโตคอลด้านฮาร์ดแวร์แบบอีเทอร์เน็ต (Ethernet) และไอพี (IP) ควบคุมการส่งแพ็กเก็ตข้อมูล

โดยกำหนดตัวอย่างการทำงานของไดรเวอร์ขึ้น ให้โมดูลของไดรเวอร์ `spnd` (Special Network Driver) รับเป็นอิสระจากอุปกรณ์ฮาร์ดแวร์ และจำลองเส้นทางการส่งผ่านข้อมูลระหว่างแอดเดรสต้นทางและปลายทางเสมือนว่าอยู่คนละเครื่อง โดยการแก้ไขแอดเดรสของเครื่องต้นทางและปลายทาง ซึ่งแพ็กเก็ตที่ส่งออกจากอุปกรณ์ที่ชื่อ `nd0` จะไปปรากฏที่อุปกรณ์ชื่อ `nd1` เสมือนส่งผ่านเน็ตเวิร์ก

5.7.1 การส่งแพ็กเก็ตข้อมูล

เมื่อเคอร์เนลต้องการส่งแพ็กเก็ตข้อมูล จะทำการเก็บแพ็กเก็ตข้อมูลลงในช็อกเก็ต (struct sk_buffer) โดยข้อมูลที่อยู่ในช็อกเก็ตบัฟเฟอร์จะเป็นเฟรม (Frame) ข้อมูลตามมาตรฐานของการส่งข้อมูลในระดับต่ำ (Physical Layer) ดังนั้น ไพล์ดริวจะมองช็อกเก็ตเป็นเพียงแพ็กเก็ตข้อมูล และลินุกซ์จะใช้ลิสต์ของโครงสร้างข้อมูล sk_buffer เดียวกันสำหรับการรับและส่งข้อมูลภายใน Network Subsystem

ตัวอย่างโค้ดต่อไปนี้ เป็นฟังก์ชันที่ใช้สำหรับควบคุมฮาร์ดแวร์เพื่อส่งข้อมูลจริง ๆ โดยจะแยกออกมาต่างหากเนื่องจากฟังก์ชันขึ้นกับอุปกรณ์ที่ใช้งานด้วย (Device specific) ซึ่งจะต้องตรวจสอบจุดที่จำเป็นก่อนแล้วค่อยเริ่มส่งข้อมูล

```
int spnd_tx(struct sk_buffer *skb, struct device *dev)
{
    int len, retval = 0;
    char *data;

    if (dev -> tbusy) {
        retval = -EBUSY;
        goto tx_done;
    }

    if (skb == NULL) {
        PDEBUG("tint for &p\n", dev);
        dev_tint(dev);
        return 0;      /* We are ready to transmit */
    }
    dev -> tbusy = 1;    /* Transmission is busy */

    /* Minimum len */
    len = ETH_ZLEN < skb -> len ? skb -> len : ETH_ZLEN;
    data = skb -> data;
    dev -> trans_start = jiffies;    /* Save the timestamp */

    /* actual deliver of data id device-specific, and not shown here */
    spnd_hw_tx(data, len, dev);

    /* Release it */
    tx_done: dev_kfree_skb(skb, FREE_WRITE);
    return retval;
}
```

ถ้าการส่งข้อมูลทำได้สำเร็จ จะต้องทำการเคลียร์ค่าฟิลด์ dev -> tbusy เพื่อบอกว่าว่างแล้ว

5.7.2 การรับแพ็กเก็ตข้อมูล

ต้องมีการจองพื้นที่ของหน่วยความจำ (sk_buffer) เพื่อทำการเก็บข้อมูลที่ได้รับมาจากอุปกรณ์เน็ตเวิร์ก ซึ่งวิธีที่มีประสิทธิภาพมากที่สุดในการทำงาน คือ การอินเทอร์รัพต์ (Interrupt)

ตัวอย่างฟังก์ชัน spnd_rx ต่อไปนี้ เป็นการรับอาร์กิวเมนต์เพื่อเป็นพอยน์เตอร์ชี้ไปยังข้อมูลของอุปกรณ์และความยาวของข้อมูล แล้วจะทำการส่งข้อมูลไปยังโปรโตคอลอีกที โดยจะทำฟังก์ชันนี้หลังจากรับข้อมูลจากอุปกรณ์เน็ตเวิร์กมาเก็บในบัฟเฟอร์แล้ว

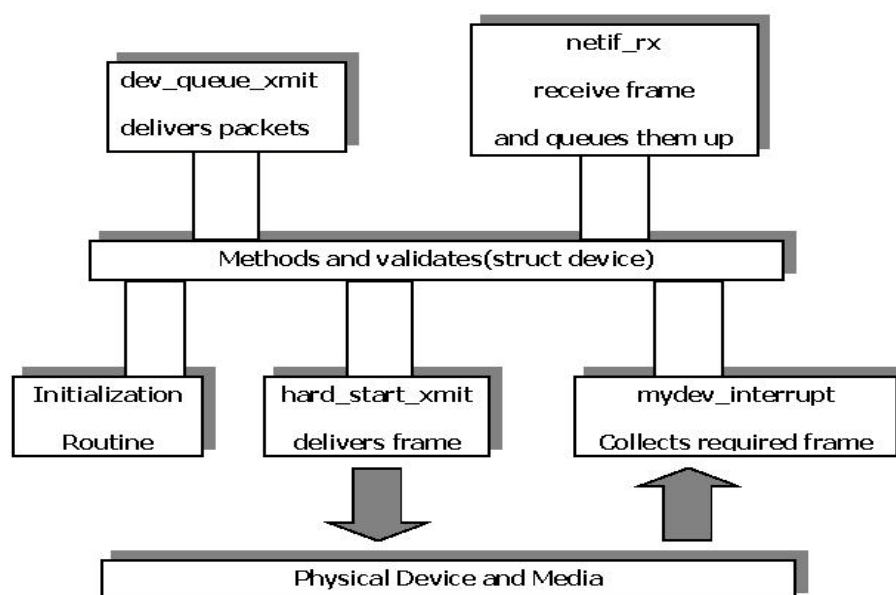
```
void spnd_rx(struct device *dev, int len, unsigned char *buf)
{
    struct sk_buffer *skb;
    struct spnd_priv = (struct spnd_priv *) dev -> priv;

    /* The packet has been retrieved from the transmission medium, Build
    an skb so the upper layers can handle it */

    skb = dev_alloc_skb(len + 2);
    if (!skb) {
        printk("spnd rx: low on mem\n");
        return;
    }
    memcpy(skb_put(skb, len), buf, len);

    /* Write metadata nad then pass to the receive level */
    skb->dev = dev;
    skb->protocol = eth_type_trans(skb, dev);
    skb->ip_summed = CHECKSUM_UNNECESSARY;
    privp -> starts.rx_packets++;
    netif_rx(skb);
    return;
}
```

เมื่อฟังก์ชันทำการจองพื้นที่บนหน่วยความจำสำหรับโครงสร้างข้อมูล skb เรียบร้อยแล้ว ฟังก์ชันจะทำการคัดลอกข้อมูลจากบัฟเฟอร์ไปยัง skb โดยใช้ฟังก์ชัน memcpy สำหรับฟังก์ชัน skb_put จะทำการอัปเดตพอยน์เตอร์ที่ชี้จุดสิ้นสุดของข้อมูลภายใน skb และความยาวข้อมูล (len) หลังจากคัดลอกข้อมูลเสร็จแล้ว และขั้นตอนสุดท้าย ไดรเวอร์จะการอัปเดตจำนวนแพ็กเก็ตที่รับได้ในโครงสร้างข้อมูล stats ซึ่งได้เก็บสถิติเกี่ยวกับเน็ตเวิร์กด้านต่าง ๆ ไว้ จากนั้นจะส่งบัฟเฟอร์ skb ไปยังโปรโตคอลชั้นถัดไป



รูปที่ 5-3: โครงสร้างในการรับและส่งแพ็กเก็ตข้อมูลของลินุกซ์